

High-Level Design Report

1. Introduction

1.1 Purpose of the system

The Multi-Agent AI Orchestration Framework is a production-ready system designed to unify multiple specialized artificial intelligence capabilities under a single, intuitive chat-based interface. The primary purpose is to enable users to accomplish complex tasks spanning multiple domains such as code generation, repository analysis, web research, text to speech generation, speech to text generation, GitHub operations, Google Workspace automation, data analysis, and vision processing through natural language interaction with a central orchestrating agent.

The system addresses the growing complexity of AI-powered workflows by abstracting the intricacies of model selection, provider management, and multi-step task execution behind a unified conversational interface. Rather than requiring users to understand which AI model or tool is best suited for a particular task, the MainAgent intelligently analyzes user intent and delegates work to the most appropriate specialized agent.

A key architectural principle of the system is provider-agnostic model execution. Users can configure each agent independently to use:

- Self-hosted local models via Ollama, LM Studio, LocalAI, or Llama.cpp
- Cloud-based providers such as OpenAI, Google, Anthropic, Mistral, Grok or Groq
- User's own custom-trained models hosted on any OpenAI-compatible server, giving researchers and developers full control over their AI backends

This flexibility empowers organizations and individual developers to balance cost, privacy, and performance according to their specific requirements. For instance, a user might configure the Coding Agent to leverage a cloud-hosted Anthropic Claude Opus 4.5 for complex reasoning tasks requiring state-of-the-art capabilities, while simultaneously running the Search Agent on an on-premise Ollama instance with quantized models optimized for faster inference on local hardware significantly reducing latency and operational costs. The system's provider abstraction layer ensures seamless switching between any backend without requiring modifications to individual agent logic or workflows.

Key purposes include:

Task Unification: Consolidate diverse AI capabilities (coding, search, GitHub, vision, data analysis) into a single access point

Provider Agnosticism: Enable seamless switching between cloud providers (OpenAI, Google, Anthropic, Mistral, Groq) and self-hosted solutions (Ollama, LM Studio, LocalAI, llama.cpp)

Transparency: Provide full visibility into agent decision-making through execution traces

Extensibility: Support addition of new specialized agents without core system modifications

Personalization: Allow per-user configuration of providers, models, and API credentials

1.2 Design goals

The architectural design of this framework is driven by the following goals:

Modularity and Extensibility

- New agents can be added by following a standardized 4-file pattern without modifying core orchestration logic
- Self-registration pattern allows agents to declare their capabilities at module load time
- Plugin-based architecture supports runtime discovery of available agents

Provider Abstraction

- Complete decoupling of agent logic from LLM provider implementations
- Factory-based abstraction supporting 11+ LLM backends
- Per-agent provider configuration enabling heterogeneous model deployment

Real-Time Observability

- Streaming architecture providing immediate feedback during agent execution
- Structured execution traces capturing agent decisions, tool invocations, and timing
- Visual representation of the agent/tool execution hierarchy

Security and Privacy

- Encrypted storage of user API credentials (AES-256-GCM)
- Session-based authentication with support for guest and regular users
- Rate limiting to prevent abuse and manage resource consumption

Scalability

- Stateless agent execution enabling horizontal scaling
- Serverless-compatible database architecture (Neon PostgreSQL)
- Resumable streams for handling interrupted connections

User Experience

- Responsive, modern interface with real-time streaming updates
- Intuitive agent selection with searchable combobox
- Configurable appearance themes, animations, layouts

Self-Hosted Capability

- Full support for on-premise LLM deployment via Ollama, LM Studio, LocalAI, or Llama.cpp and OpenAI-compatible servers
- Minimal cloud dependencies for privacy-conscious deployments

1.3 Definitions, acronyms, and abbreviations

Agent: An autonomous AI module designed to handle specific types of tasks, with its own system prompt, tool set, and configuration

MainAgent: The central orchestrator responsible for intent analysis and task delegation to specialized agents

BaseAgent: Abstract base class providing shared runtime functionality for all sub agents

Tool: A structured callable capability with defined schema that agents can invoke to perform actions

LangGraph: A library for building stateful, multi-step agent workflows using graph-based state machines

LangChain: A framework for developing LLM-powered applications with abstractions for chains, agents, and tools

MCP: Model Context Protocol - a standardized protocol for AI model interaction with external tools and services

RAG: Retrieval-Augmented Generation - enhancing LLM responses by retrieving relevant context before generation

LLM: Large Language Model - neural networks capable of understanding and generating human-like text

pgvector: PostgreSQL extension for vector similarity search, used for RAG embeddings

HNSW: Hierarchical Navigable Small World - an algorithm for approximate nearest neighbor search

JWT: JSON Web Token compact, URL-safe means of representing claims for authentication

SSE: Server-Sent Events - web technology for pushing real-time updates from server to client

ORM: Object-Relational Mapping - technique for converting data between type systems

Drizzle: TypeScript-first ORM used for database operations

NextAuth: Authentication library for Next.js applications

Neon: Serverless PostgreSQL database service

Ollama: Tool for running large language models locally with OpenAI-compatible API

Groq: Cloud provider offering high-speed, low-latency LLM inference

E2B: Sandboxed code execution environment for running untrusted code safely

1.4 Overview

The Multi-Agent AI Orchestration Framework is a web-based platform that enables users to interact with multiple specialized AI agents through a unified chat interface. At its core, a central orchestrator (MainAgent) receives natural language requests, analyzes user intent, and delegates tasks to the most appropriate specialized agent.

The system follows a layered architecture with clear separation between user interface, API handling, agent orchestration, and data persistence. Each specialized agent operates as an autonomous module with its own reasoning capabilities and tools, while the orchestration layer manages coordination and response aggregation.

The framework is designed to be provider-agnostic, allowing each agent to use different LLM backends ranging from cloud services to self-hosted models. This flexibility enables organizations to balance cost, performance, and privacy according to their specific requirements.

Key characteristics of the system include real-time streaming responses, transparent execution tracing, extensible agent architecture, and secure credential management for external service integrations.

2. Current software architecture

The system is already implemented as a Next.js 15 (App Router) application with a server-side multi-agent orchestration layer. The architecture centers on a MainAgent that dynamically delegates tasks to specialized agents (GitHub, Google Workspace, Search, Codebase, Vision, Frontend, etc.) and streams intermediate tool and agent events back to the UI. User-specific LLM settings and secrets are stored in PostgreSQL with encryption, and the system can be configured to use both cloud and self-hosted inference backends.

Architecture layers and responsibilities

Frontend Layer:

components/chat.tsx renders the chat UI, consumes streamed agent output in real time, visualizes agent and tool execution flow, and applies UI customization actions such as themes, colors, and effects. The UI supports multimodal inputs including text with optional image or file upload and agent or model selection.

API Layer:

Next.js API routes provide the system's public execution entry points. POST /api/chat orchestrates agent execution and persistence while streaming results. POST /api/files/upload handles uploads. /api/user_dashboard/* manages per-user agent configuration and enablement. /api/agents_dashboard/metrics exposes observability metrics derived from stored execution traces.

Agent Orchestration Layer:

lib/agents/mainAgent constructs a LangGraph at runtime from the agentRegistry and delegates tasks via generated delegation tools. All agents inherit from BaseAgent, which encapsulates LLM instantiation, tool binding, event streaming, and runtime graph caching. Specialized agents integrate with MCP servers, Google Workspace APIs, and web search and scraping services through their tool sets.

Provider Abstraction Layer:

lib/agents/llmFactory.ts implements a factory pattern to instantiate provider-specific chat models and to target either cloud LLM APIs or self-hosted inference endpoints such as Ollama. Per-agent runtime overrides are resolved from the database and merged at execution time. Graphs are cached using a configuration version key to avoid unnecessary reinitialization.

Data Persistence Layer:

PostgreSQL stores users, chats, messages, agent configurations and preferences, and documents. Database access is implemented via Drizzle ORM for type-safe schema definitions, query construction, and migrations. Semantic code retrieval is backed by pgvector embeddings with an HNSW index. Uploaded files are stored in object storage (Vercel Blob in the current build) and referenced by URL.

Security & Authentication :

NextAuth v5 with regular and guest modes, JWT-based session handling, and middleware-based access control are used. Sensitive credentials are stored encrypted using AES-256-GCM.

LLM provider abstraction and deployment

The LLM layer is abstracted through a provider factory (lib/agents/llmFactory.ts) that can instantiate different backends including Google Gemini, OpenAI-compatible APIs, Anthropic, Groq, Mistral, and self-hosted providers such as Ollama. Each agent ships

with a default provider and model configuration, but users can override this per agent via the Settings UI. Overrides such as provider, model, API key, base URL, and agent-specific secrets are stored in PostgreSQL and merged at runtime before agent execution.

During orchestration with MainAgent, the API layer can resolve configurations for multiple agents and pass per-agent overrides so delegated agents run with the user's selected provider and model. Agent-specific secrets can be merged and forwarded at runtime to enable authenticated tool invocations without hardcoding credentials.

The runtime also applies lightweight caching. Agents track a configuration version key and only recreate their LLM and tools graph when user configuration changes, reducing latency and repeated initialization overhead.

Agent layer

MainAgent: Orchestrator that delegates tasks using the registry.

GitHubAgent: GitHub MCP integration for repositories, issues, pull requests, commits, and files.

GoogleWorkspaceAgent: Gmail, Calendar, Drive, Docs, Sheets, and Slides integration via Google REST APIs.

SearchAgent: Web search, scraping, and academic search through Exa, DuckDuckGo, arXiv, and Semantic Scholar.

CodebaseAgent: Semantic code search using pgvector embeddings with an indexer that builds embeddings.

CodingAgent: Code implementation assistant. Default LLM deployment is self-hosted via Ollama (glm-4.7-flash) but is reconfigurable and can call semantic retrieval when repository context is needed.

VisionAgent: Image analysis via Gemini Vision and image generation via Hugging Face Inference API.

FrontendAgent: Emits UI actions for themes, colors, fonts, and visual effects.

HuggingFaceAgent: MCP-based model, dataset, and paper discovery.

Runtime request flow

1. The user sends a message in the chat UI with text and optional file or image.
2. `/api/chat` converts UI messages to `AgentChatMessage` format and resolves user-specific agent configurations and secrets from the database.
3. The selected agent or main-agent executes via `LangGraph` and tool and agent events are streamed back.
4. The UI renders streaming responses and execution flow, and generated artifacts or images are stored and referenced.
5. Messages, artifacts, and execution metadata are persisted in PostgreSQL.

Agent framework and orchestration details

The agent runtime is implemented with `LangGraph`. All agents inherit from `BaseAgent`, which builds a compact execution graph consisting of an LLM node and a Tool node. The LLM receives the system instruction and conversation history. If it returns tool calls, execution is routed to the Tool node and then back to the LLM until completion.

`MainAgent` extends this pattern by creating a dynamic orchestration graph from the `agentRegistry`. Each specialized agent self-registers at module load time with metadata including agent id, delegation tool name, and compiled subgraph. The `MainAgent` exposes one delegation tool per registered agent and routes the conversation to the corresponding sub-agent subgraph.

Streaming, tracing, and persistence

Agents emit newline-delimited JSON events such as `agent_started`, `agent_stream`, `tool_started`, `tool_ended`, `agent_ended`, and `agent_error`. The `/api/chat` route forwards these chunks to the client while also constructing a hierarchical execution trace structure. The trace is stored as a message part, enabling the UI to render both the final answer and the step-by-step execution flow.

Multimodal inputs and file handling

The UI supports image and file uploads. Files are uploaded through `/api/files/upload` to object storage and referenced by URL inside the message. The `/api/chat` route injects these URLs into agent message content in a standardized format so multimodal agents can operate directly on uploaded assets.

Operational monitoring (Agents Dashboard)

The Agents Dashboard computes aggregate metrics such as agent and tool call counts, latency, and error rates by scanning stored execution traces from recent assistant messages. This provides lightweight observability without requiring a separate tracing backend.

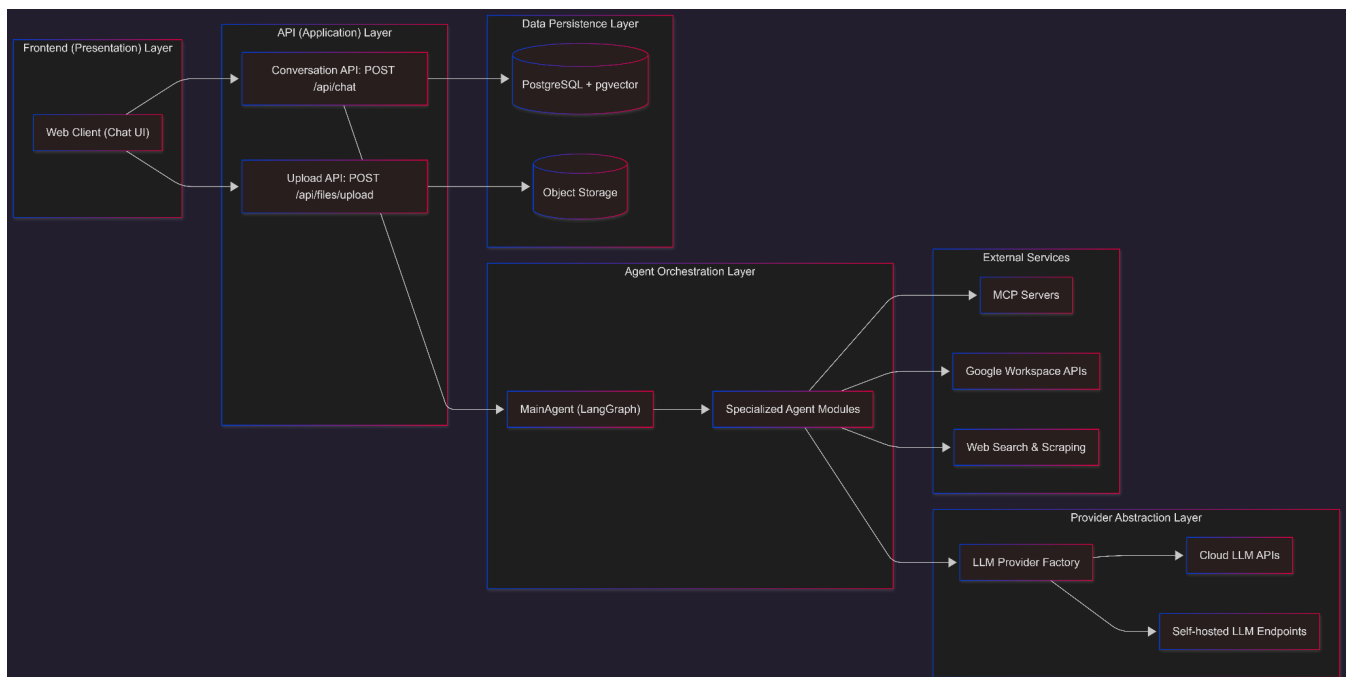
Codebase retrieval (RAG) path

The system implements semantic code retrieval backed by PostgreSQL pgvector. This retrieval capability is primarily used by CodebaseAgent and can also be invoked by CodingAgent when additional repository context is required.

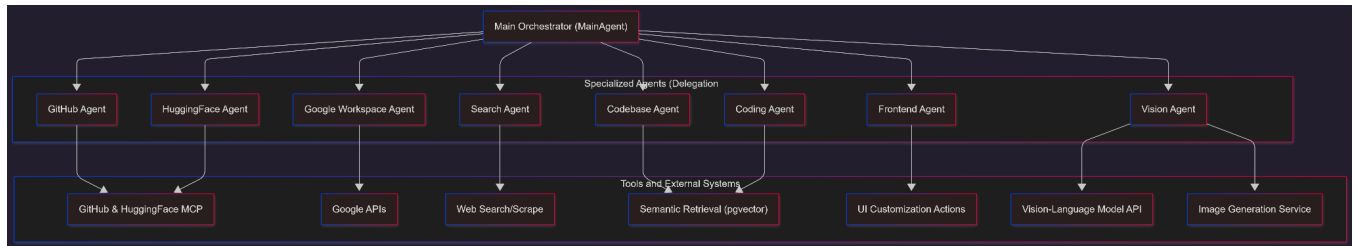
An indexing script scans the repository, chunks TypeScript and JavaScript sources using the TypeScript AST, generates embeddings, and stores them with an HNSW index for fast cosine similarity search. The shared search_codebase tool queries these embeddings and returns relevant code snippets to ground responses. In the current build, embeddings are generated using Google text-embedding-004 with 768 dimensions.

Current architecture diagrams

(A) System context diagram



(B) Agent orchestration diagram



3. Proposed software architecture

3.1 Overview

The proposed architecture maintains the current multi-agent orchestration paradigm while introducing enhancements for scalability, observability, and extensibility. The system is structured as a layered architecture with clear separation of concerns between presentation, business logic, data access, and external integrations.

The architecture follows a layered, agent-oriented design as it has been shown in the Current Software Architecture section.

3.2 Subsystem decomposition

- 1. MainAgent Subsystem:** Acts as the central orchestrator responsible for high-level intent detection and workflow routing. It analyzes user requests and delegates tasks to the appropriate sub-agent (e.g., routing code queries to CodingAgent or repository questions to CodebaseAgent).
- 2. SubAgent Subsystem:** A suite of specialized agents extending a common BaseAgent abstract class, powered by LangGraph for state management:
 - Development Agents:** CodingAgent (general programming), CodebaseAgent (repo analysis), GithubAgent (issue/PR management), FrontendAgent (UI generation).
 - Productivity Agents:** GoogleWorkspaceAgent (Docs/Drive integration), SearchAgent (web research), EmailAgent (communication).
 - Multimodal Agents:** VisionAgent (image analysis/generation), VoiceAgent.
 - Data Agents:** DataAnalystAgent (visualization and analysis).
- 3. Execution & Provider Subsystem:** Manages model inference through a unified factory pattern (createLLM). It supports a hybrid execution environment defined by LLMProvider types:

Local Inference: Privacy-focused execution via Ollama, Llama.cpp, or LocalAI (running on CPU/GPU).

Cloud Inference: Scalable execution via OpenAI, Anthropic, Google Gemini, or Groq.

Configuration: Uses LLMImplMetadata to dynamically switch providers and parameters per agent.

4. **User Interface Subsystem:** A Next.js-based frontend responsible for multimodal interaction. Key components include:
 - Chat Interface: Handles streaming text deltas and tool execution logs via use-chat hooks.
 - Artifacts System: Renders structured outputs like code (codeDelta), images (imageDelta), and spreadsheets (sheetDelta) in interactive panels.
 - Multimodal Input: Supports text, image uploads, and voice input.
5. **Logging & Monitoring Subsystem:** Implements an event-driven telemetry system. The BaseAgent captures and streams lifecycle events (AGENT_STARTED, TOOL_STARTED, AGENT_STREAM, AGENT_ENDED) to the frontend and server logs for real-time observability and debugging.

3.3 Hardware/software mapping

The system adopts a hybrid cloud and local computing model, leveraging serverless infrastructure for web hosting and database management, while offering flexibility for AI inference execution. The mapping of software components to hardware resources is defined as follows:

Client-Side Hardware:

Device: Any standard personal computer or mobile device with a modern web browser.

Software: The frontend application runs within the browser environment (Chrome, Firefox, Edge, Safari), executing the **React**-based UI and handling real-time Server-Sent Events (SSE) for streaming responses.

Server-Side Infrastructure:

Runtime Environment: **Next.js 15** hosted on a serverless Node.js runtime or a containerized Node.js environment.

Responsibility: Handles API routes, authentication (NextAuth), LangGraph orchestration, and secure communication with AI providers.

Data Persistence Layer:

Database: Neon (Serverless PostgreSQL) running on cloud infrastructure. This handles structured data (Users, Chats, Agent Configs) and vector embeddings (pgvector) for RAG operations.

Object Storage: Vercel Blob storage for unstructured data such as uploaded user files, images, and generated artifacts.

AI Inference Infrastructure:

Cloud Inference: API-based interaction with external providers (OpenAI, Anthropic, Google) running on provider-managed GPU clusters.

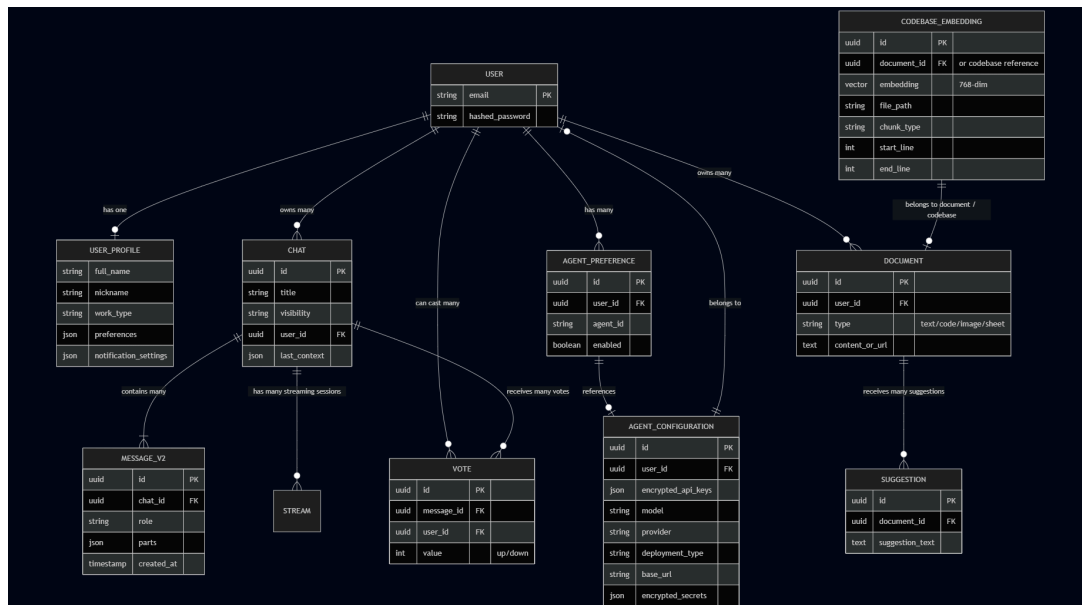
Local Inference (Self-Hosted): Users may opt to run models locally using **Ollama**, **LM Studio**, or **LocalAI**. In this scenario, the inference utilizes the user's or organization's local CPU/GPU resources (e.g., NVIDIA RTX series or Apple Silicon M-series chips) to process LLM requests without data leaving the local network.

3.4 Persistent data management

The data management system of the multi-agent AI framework is centered on PostgreSQL with Drizzle ORM for all structured data, complemented by Vercel Blob Storage for handling large files and media.

- User authentication data, including hashed passwords and profile information, resides in dedicated User and UserProfile tables, while chat conversations are preserved in Chat and Message_v2 tables with support for attachments and voting feedback.
- Agent configurations, including encrypted API keys and model selections, are securely stored in AgentConfiguration tables using AES-256-GCM encryption with script-derived keys, ensuring sensitive credentials like GitHub PATs and HuggingFace tokens remain protected.
- Generated artifacts such as code, images, and documents are stored in the Document table for text-based content, with larger binary files automatically routed to Vercel Blob Storage and referenced via URLs.
- The system also maintains vector embeddings for codebase analysis in a specialized CodebaseEmbedding table with HNSW indexing for efficient similarity searches, while user preferences and agent enablement settings are tracked in AgentPreference tables.

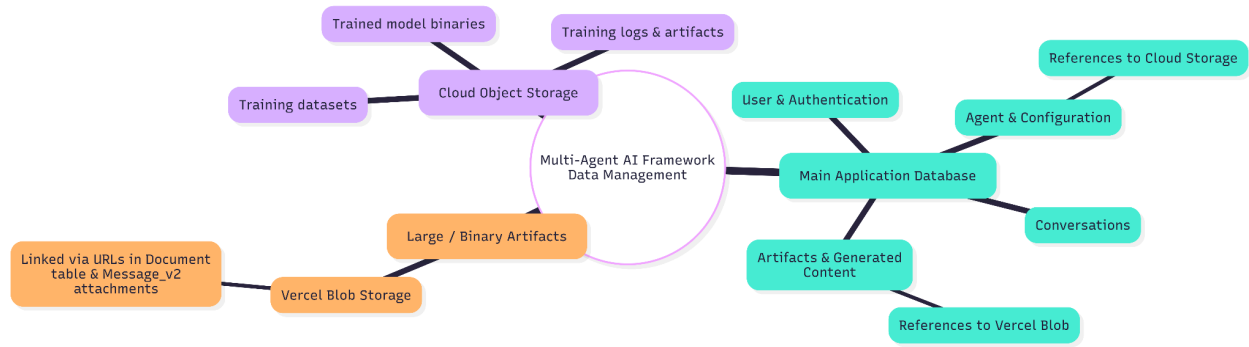
This hybrid approach balances database efficiency for structured queries with cloud scalability for media assets, all while maintaining data integrity through migrations and comprehensive encryption.



The framework's data management for trained or fine-tuned models consists of two parts: training data management which is accessible by developers and trained model storage which is accessible by users.

Training data can be stored separately from the framework in cloud storage (Google Cloud, AWS S3...) with developer controlled access and versioning, ensuring sensitive datasets stay isolated from user data. Trained model binaries and checkpoints can be stored in cloud object storage with CDN distribution, then they can be integrated into the existing AgentConfiguration table through new model registry entries that track versions, storage URLs, performance metrics, and compatibility information, or a new separate table can be created for the trained models.

Additional database tables may manage model metadata without exposing training details to users, while the framework's encryption system extends to protect model access credentials. An overview of the data management is shown in Figure...



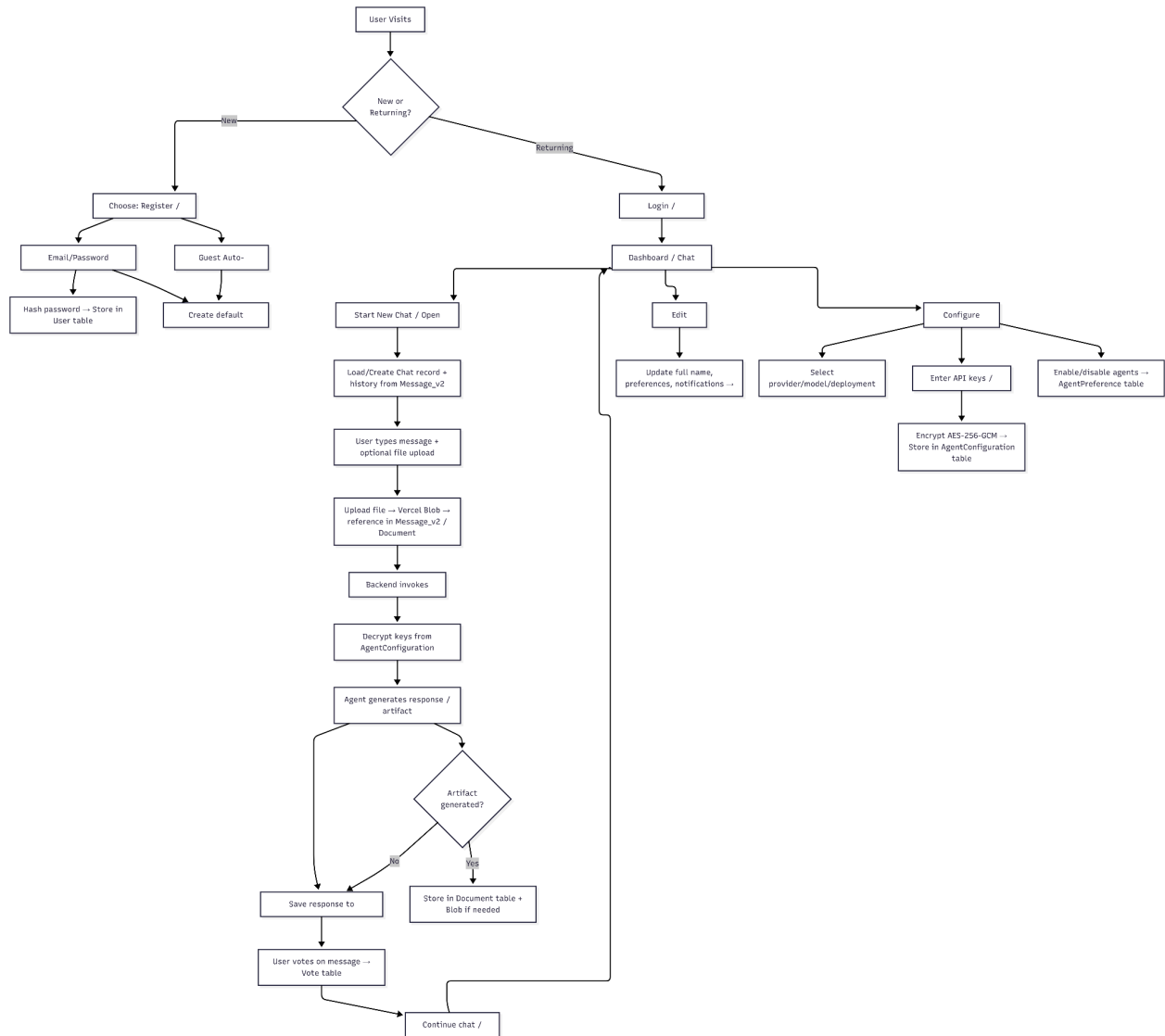
A usual user flow is designed as follows:

A new user arrives at the Multi-Agent AI Framework for the first time, greeted by a clean login screen offering email/password registration or guest mode. Choosing to register, the user enters an email address and password; the system hashes the password securely and creates a new record in the User table, while simultaneously generating a default UserProfile entry for future personal details. After successful signup, the user is welcomed to the dashboard and prompted to complete their profile—adding a full name, nickname, work preferences, and notification settings—which are saved directly to the UserProfile table.

Next, the user explores the agent configuration section, where they enable or disable specific agents via toggles (recorded in the AgentPreference table) and begin setting up their preferred language models: selecting providers such as Google, Groq, or OpenAI, choosing models, and entering API keys or other secrets. These sensitive values are encrypted on the server using AES-256-GCM before being stored in the AgentConfiguration table, linked to the user’s account for secure, on-demand retrieval during future sessions.

Satisfied with the setup, the user starts their first chat by clicking “New Chat.” The system creates a fresh Chat record associated with their user ID, and as the user types their initial message (optionally attaching an image or document), the frontend uploads any large files to Vercel Blob storage and references the URLs in the Message_v2 table alongside the text content. The backend loads the relevant agent configuration, decrypts the necessary keys, invokes the chosen agent(s), and streams the response back—saving both the user’s input and the agent’s reply as new entries in Message_v2. If the agent generates an artifact (code snippet, generated image, or document), it is either stored as text in the Document table or uploaded to Blob with a reference added.

Throughout the conversation the user can upvote helpful messages, with each vote recorded in the Vote table, ensuring personalized feedback persists. By the end of the session, the user’s entire history, profile, agent preferences, and configurations remain securely stored and ready to be reloaded the next time they return—creating a seamless, stateful experience from the very first interaction. Corresponding user flow chart can be seen in below.



3.5 Access control and security

Security is a foundational pillar of the framework, ensuring strict data isolation and the protection of sensitive user credentials. The system implements a defense-in-depth strategy:

Authentication & Session Management:

The system utilizes **NextAuth v5** to handle authentication flows, supporting both credential-based login (Email/Password with bcrypt hashing) and OAuth providers.

Sessions are managed via secure, HTTP-only **JWT (JSON Web Tokens)**, ensuring stateless and scalable verification of user identity across API requests.

Middleware enforcement prevents unauthorized access to protected routes such as `/api/chat` and `/api/user_dashboard`.

Credential Encryption (Secrets Management):

Sensitive user data, specifically API keys (e.g., OpenAI Key, GitHub PAT) and agent-specific secrets, are never stored in plain text.

The system employs **AES-256-GCM encryption** for all sensitive fields in the PostgreSQL database.

Decryption keys are managed via server-side environment variables (`process.env`), ensuring that even a database dump does not compromise user credentials. Keys are decrypted only momentarily at runtime within the server memory to instantiate the LLM client.

- **Data Isolation & Privacy:**

Logical Isolation: Database queries are scoped strictly by `userId`. Users can only access chat history, artifacts, and configurations associated with their own account.

Sanitization: All inputs are sanitized to prevent SQL injection and XSS attacks.

Provider Privacy: When using local inference, data remains within the user's infrastructure, adhering to strict privacy requirements.

- **Operational Security:**

Rate Limiting: API routes implement rate limiting to prevent abuse and Denial-of-Service (DoS) attacks.

Input Validation: Strict schema validation (using Zod) is applied to all incoming API requests to ensure data integrity before processing.

3.6 Global software control

Global control of the software is managed through an event driven, graph based orchestration engine. The control flow is not linear but cyclic, governed by the **LangGraph** state machine architecture.

Central Orchestrator:

The MainAgent serves as the root controller. Upon receiving a user request, it does not immediately execute a task but instead analyzes the Intent.

Based on the intent, the control flow is dynamically routed to a specialized subgraph or a specific tool node.

State Management:

The entire conversation state is immutable at each step but evolves through a sequence of state transitions (Node -> Edge -> Node).

The application state includes the conversation history, the current "scratchpad" of agent thoughts, and the status of tool executions.

Execution Lifecycle:

Request Initiation: The UI triggers a POST request to /api/chat.

Context Loading: The controller loads the user's encrypted configuration and hydrates the agent graph.

Graph Traversal: The engine traverses the graph nodes. If a node requires external input (e.g., a tool call result), the execution pauses or loops until the tool output is injected back into the state.

Streaming Response: Control signals and text chunks are streamed back to the client in real-time using the NDJSON format, providing immediate feedback (e.g., "Browsing web...", "Generating code...") before the final response is complete.

Error Handling & Recovery:

The control layer includes global error boundaries. If a sub-agent fails or an API times out, the MainAgent catches the exception, logs the trace, and attempts a graceful fallback or informs the user without crashing the entire session.

3.7 Boundary conditions

The multi-agent AI framework operates within several critical boundary conditions that define its operational scope and limitations. These boundaries collectively ensure system stability, security, and predictable scaling while maintaining the framework's multi-agent orchestration capabilities within manageable operational constraints.

At the user level, boundaries ensure data isolation where each user's chat history, agent configurations, and artifacts remain strictly segregated, with encrypted API keys and preferences stored per-user but never shared across accounts.

System boundaries limit agent capabilities to predefined tool sets and external API integrations, preventing unrestricted access to user systems or sensitive data beyond configured permissions.

Resource boundaries constrain the framework to PostgreSQL database limits (approximately 512MB-4GB depending on Neon tier), Vercel Blob storage quotas, and AI provider rate limits (OpenAI's 10,000 RPM, Anthropic's 50 requests/minute), while model size boundaries restrict self-hosted deployments to GPU memory constraints.

Security boundaries enforce AES-256-GCM encryption for all sensitive data with environment-variable controlled keys, while operational boundaries limit concurrent users based on database connection pools and require manual model training by developers rather than end users.

Geographic boundaries may apply through data residency requirements of cloud providers, and cost boundaries emerge from API usage metering that could restrict heavy users.

4. Subsystem services

MainAgent:

run(messages): Entry point for handling user requests and orchestrating the conversation flow.

agentNode: Analyzes conversation history to determine the intent and routes to the appropriate sub-agent or responds directly.

createTools: Initializes orchestration tools (if any) for the session.

SubAgents (via BaseAgent & LangGraph):

run(messages, config, secrets): Executes the agent workflow, managing the event stream (started, streaming, ended).

agentNode: Invokes the specific LLM with system instructions and tools; handles fallback logic for empty responses.

createTools: Registers domain-specific tools (e.g., GitHub API, File Ops) with runtime secrets.

getOrCreateRuntimeGraph: Manages caching of the compiled LangGraph runnable for performance.

agentRoute: Determines the next step in the graph (e.g., execute a tool or end turn) based on LLM output.

Execution & Provider Service:

createLLM: Factory service that instantiates the appropriate BaseChatModel based on LLMProvider configuration (local or cloud).

bindTools: Binds the agent's dynamic structured tools to the LLM instance.

UI Subsystem:

useChat / useArtifact: React hooks for managing chat state and artifact visibility.

Server Actions: Handles authentication and initiates agent streams.

Artifact Renderer: Renders specialized UI components (Code, Sheet, Image) based on streaming data types.

Logging & Monitoring Service:

Event Streaming: Streams structured events (AGENT_STARTED, TOOL_STARTED, AGENT_ERROR) to the client for real-time feedback.

Console Logging: Server-side logging of agent lifecycle events, tool inputs/outputs, and error traces for debugging.

5. Glossary

Term	Definition
Agent	An autonomous AI module with specific capabilities, system prompt, and tool set
Agent Registry	Dynamic catalog of available agents enabling runtime discovery
API Key	Secret credential for authenticating with external services
App Router (Next.js)	Next.js routing model based on the app/ directory (server-first routing and Route Handlers)
Artifact	Generated content (code, text, image, spreadsheet) stored as documents
Base URL	Endpoint URL for LLM provider API, configurable for self-hosted models
BaseAgent	Abstract class providing shared functionality for all agents (LLM binding, tool execution, streaming, caching)
bcrypt	Password hashing algorithm used for credential storage
BYOM	Bring Your Own Model - user-configured model endpoints
Configuration Version Key	Cache key derived from runtime LLM config + secrets to reuse or rebuild agent graphs safely
Delegation	Process of MainAgent forwarding tasks to specialized agents

Drizzle ORM	TypeScript-first object-relational mapper for PostgreSQL
Embedding	Vector representation of text/code for semantic similarity
Execution Trace	Structured log of agent decisions and tool invocations (persisted as ExecutionStep[])
HNSW	Hierarchical Navigable Small World - vector search algorithm used for approximate nearest neighbor retrieval
Intent Analysis	Process of determining user request type
JWT	JSON Web Token for session management
LangChain	Framework for LLM application development
LangGraph	Library for stateful agent workflows
LLM	Large Language Model
LLM Factory	Factory pattern for provider-agnostic LLM instantiation
MainAgent	Central orchestrator for task delegation
MCP	Model Context Protocol for AI-tool interaction
Neon	Serverless PostgreSQL provider
Newline-delimited JSON (NDJSON)	Streaming format where each event is a JSON object separated by newline characters

NextAuth / Auth.js	Authentication library for Next.js
Object Storage (Vercel Blob)	Storage for uploaded files and generated media; assets are referenced by URL
Ollama	Local LLM inference runtime
pgvector	PostgreSQL extension for vector operations
Provider	Service hosting LLM inference (cloud or self-hosted)
RAG	Retrieval-Augmented Generation
Route Handler (Next.js)	Server endpoint implemented as route.ts under app/
Runtime Secrets	Decrypted per-user credentials forwarded to agents/tools at execution time
Self-Hosted	Running inference on user-controlled infrastructure
Streaming	Incremental response delivery
Tool	Callable capability with defined schema
ToolNode	LangGraph node responsible for executing tool calls and returning tool outputs into the graph

6. References

- [1] T. Liu et al., "CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society," arXiv preprint arXiv:2303.17760, 2023.
- [2] R. Qiao et al., "ChatDev: Revolutionizing Software Development with AI-Collaborative Agents," arXiv preprint arXiv:2307.07924, 2023.

- [3] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv preprint arXiv:2302.04761, 2023.
- [4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629, 2022.
- [5] Y. Zeng et al., "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in HuggingFace," arXiv preprint arXiv:2303.17580, 2023.
- [6] S. Arora et al., "DSPy: A Programming Model for Large Language Models," arXiv preprint arXiv:2402.19148, 2024.
- [7] X. Zheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," arXiv preprint arXiv:2205.13701, 2022.
- [8] G. Gerganov, "llama.cpp," GitHub Repository. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [9] J. Li et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," arXiv preprint arXiv:2301.12597, 2023.
- [10] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021.
- [11] OpenAI, "Whisper: Open-source Speech Recognition," GitHub Repository. [Online]. Available: <https://github.com/openai/whisper>
- [12] T. Schick, J. Dwivedi-Yu, R. Dessì, et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv preprint arXiv:2302.04761, Feb. 2023.
- [13] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, Y. Zhuang, "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face," arXiv preprint arXiv:2303.17580, Mar. 2023.
- [14] X. Cai, X. Wang, T. Ma, X. Chen, D. Zhou, "Large Language Models as Tool Makers," arXiv preprint arXiv:2305.17126, May 2023.
- [15] Y. Qin et al., "ToolLLM: Facilitating Large Language Models to Master 16 000+ Real-world APIs," arXiv preprint arXiv:2307.16789, Jul. 2023.
- [16] W. Wang et al., "LLMind: LLM-Based Agent Framework for Task Execution with IoT Integration," arXiv preprint arXiv:2312.09007, Dec. 2023.
- [17] S. Reicher, M. Schweter, "Multi-Agent Collaboration via Evolving Orchestration," arXiv preprint arXiv:2505.19591, May 2025.
- [18] Y. Li, J. Li, X. Wang, "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and LLMs," arXiv preprint arXiv:2301.12597, Jan. 2023.
- [19] J. Wei et al., "Parallel CPU–GPU Execution for LLM Inference on Constrained GPUs," arXiv preprint arXiv:2506.03296v2, Jun. 2025.
- [20] Z. Hou, J. Tang, Y. Wang, "HALO: Hierarchical Autonomous Logic-Oriented Orchestration for Multi-Agent LLM Systems," arXiv preprint arXiv:2505.13516, May 2025.

- [21] L. Zhang, Z. Chen, et al., "MARCO: Multi-Agent Real-time Chat Orchestration," arXiv preprint arXiv:2410.21784, Oct. 2024.
- [22] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." **arXiv preprint arXiv:2210.03629**. <https://arxiv.org/abs/2210.03629>
- [23] Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." **arXiv preprint arXiv:2302.04761**. <https://arxiv.org/abs/2302.04761>
- [24] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., & Wang, C. (2023). "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." **arXiv preprint arXiv:2308.08155**. <https://arxiv.org/abs/2308.08155>
- [25] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." **arXiv preprint arXiv:2005.11401**. <https://arxiv.org/abs/2005.11401>
- [26] Malkov, Y. A., & Yashunin, D. A. (2016). "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs." **arXiv preprint arXiv:1603.09320**. <https://arxiv.org/abs/1603.09320>